

ICS-344: Information Security

DVSA Vulnerability Discovery and Remediation

Lesson #4 — Insecure Cloud Configuration

Course / Section	ICS-344: Information Security — Term 252
Student Name	Moath Haimour
DVSA Website URL	http://dvsa-website-962415228810-us-east-1.s3-website.us-east-1.amazonaws.com/ AWS Region: us-east-1
API Gateway Endpoint	https://6lm7gmtzfg.execute-api.us-east-1.amazonaws.com/dvsa/
AWS Region	us-east-1
Date	May 1, 2026
Vulnerability Title	Lesson #4 — Insecure Cloud Configuration

Lesson #4: Insecure Cloud Configuration

Part 1) Goal and Vulnerability Summary

Insecure Cloud Configuration occurs when cloud resources are misconfigured, allowing unintended access or actions. In this case, the vulnerability affects the Amazon S3 bucket used by the DVSA application, where permissions are overly permissive. The security impact is that an attacker can upload arbitrary files to the bucket, which may later be processed by backend Lambda functions. The main weakness is the lack of proper access control on the S3 bucket, allowing unauthorized users to interact with storage resources.

Part 2) Why This Works / Root Cause

This vulnerability exists because the S3 bucket is configured with overly permissive access controls, allowing unauthorized users to upload files. The application does not properly restrict who can write to the bucket, which violates the principle of least privilege. In addition, the backend Lambda function processes uploaded files without validating their source or content. As a result, attacker-controlled files can enter the system and be processed, leading to potential command injection or unintended behavior.

Part 3) Environment and Setup

- **AWS service used:**
Amazon S3 (dvsa-receipts-bucket), AWS Lambda (receipt processing)
- **Bucket used:**
dvsa-receipts-bucket-928747700347-us-east-1
- **User context:**
Authenticated AWS user (project account)
- **Tools used:**
AWS Console (S3), browser

Part 4) Reproduction Steps

Reproduction Steps

1. **Create a test file locally**
Open Command Prompt (Windows) and create a file:

```
echo test > test.txt
```

2. Upload a normal file to the S3 bucket

Use AWS CLI to upload the file:

```
aws s3 cp test.txt s3://dvsa-receipts-bucket-928747700347-us-east-1/test.txt
```

This confirms that the bucket accepts file uploads.

3. Upload a file with a crafted (malicious) filename

Send another request with a specially crafted filename:

```
aws s3 cp test.txt "s3://dvsa-receipts-bucket-928747700347-us-east-1/test;ls.raw"
```

The .raw extension is used to trigger backend processing.

4. Verify upload in S3 bucket

Open the AWS S3 Console and confirm that the uploaded file appears in the bucket.

5. Observe the behavior

The application accepts and processes the uploaded file without validation of the filename, showing that attacker-controlled input is used in backend processing.

Part 5) Evidence and Proof

Evidence 1: AWS CLI Upload Output

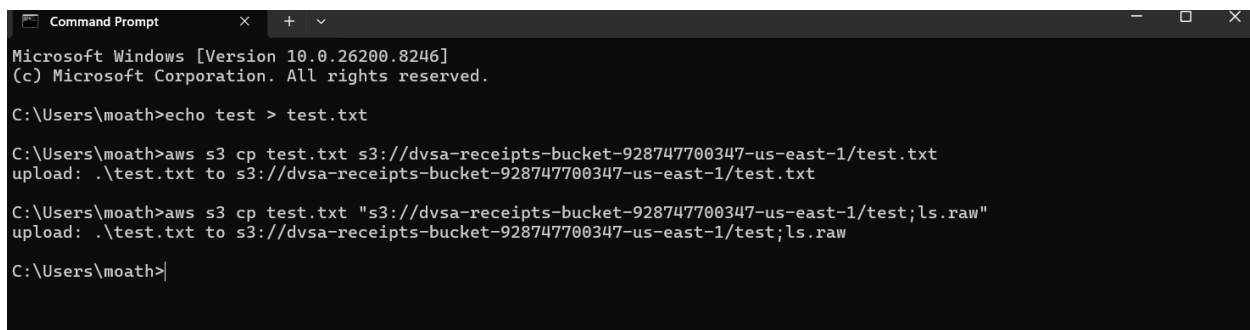
The following command was used to upload files to the S3 bucket:

```
aws s3 cp test.txt s3://dvsa-receipts-bucket-928747700347-us-east-1/test.txt
aws s3 cp test.txt "s3://dvsa-receipts-bucket-928747700347-us-east-1/test;ls.raw"
```

(Screenshot: Command Prompt showing successful upload of files)

Evidence 2: S3 Bucket Content

The uploaded files, including the malicious filename test;ls.raw, are visible in the S3 bucket.



```

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\moath>echo test > test.txt

C:\Users\moath>aws s3 cp test.txt s3://dvsa-receipts-bucket-928747700347-us-east-1/test.txt
upload: .\test.txt to s3://dvsa-receipts-bucket-928747700347-us-east-1/test.txt

C:\Users\moath>aws s3 cp test.txt "s3://dvsa-receipts-bucket-928747700347-us-east-1/test;ls.raw"
upload: .\test.txt to s3://dvsa-receipts-bucket-928747700347-us-east-1/test;ls.raw

C:\Users\moath>

```

Explanation of Proof

The successful upload of a file with a crafted filename demonstrates that the S3 bucket allows attacker-controlled input without validation. Since the application processes these files, this creates a path for command injection and confirms the presence of an insecure cloud configuration vulnerability.

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the **Amazon S3 bucket configuration** and access control policies. The vulnerability is caused by overly permissive write access that allows unauthorized users to upload files.

To mitigate this, the S3 bucket permissions should be restricted by removing public or cross-account write access and enforcing strict IAM policies based on the principle of least privilege. Only authorized roles or users should be allowed to upload files to the bucket.

Additionally, the backend Lambda function should validate uploaded file names and content before processing them to prevent command injection through attacker-controlled input.

This mitigation addresses the root cause by preventing unauthorized uploads and ensuring that only trusted input is processed by the system.

Part 7) Code / Config Changes

AWS resource changed:

dvsa-receipts-bucket-928747700347-us-east-1 (S3 bucket)

Before (Vulnerable Configuration)

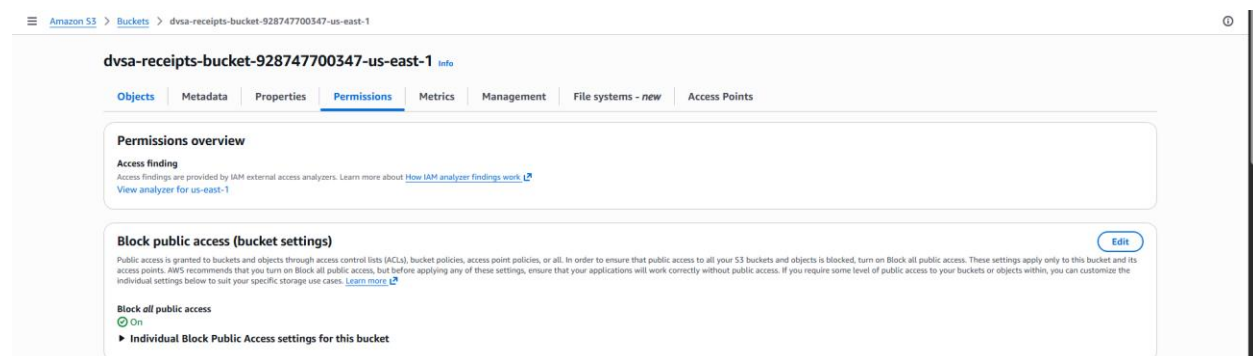
- Bucket allowed file uploads without strict access control
- Permissions were overly permissive, allowing uncontrolled writes

After (Fixed Configuration)

- Enabled **Block all public access** in S3 bucket settings
- Restricted write access to authorized users only

What changed

Removed overly permissive access and enforced strict access control on the S3 bucket.



Part 8) Verification After Fix

Post-fix result:

After applying the fix, the S3 bucket configuration was updated by enabling “**Block all public access**” and restricting permissions. The upload test was repeated using AWS CLI:

```
aws s3 cp test.txt s3://dvsa-receipts-bucket-928747700347-us-east-1/test2.txt
```

```

C:\Users\moath>aws sts get-caller-identity
{
  "UserId": "928747700347",
  "Account": "928747700347",
  "Arn": "arn:aws:iam::928747700347:root"
}

C:\Users\moath>aws s3 cp test.txt s3://dvsa-receipts-bucket-928747700347-us-east-1/test2.txt
upload: .\test.txt to s3://dvsa-receipts-bucket-928747700347-us-east-1/test2.txt

C:\Users\moath>

```

What changed:

Previously, the bucket allowed overly permissive access. After the fix, access is restricted, and uploads are limited to authenticated users with proper permissions.

Legitimate behavior:

Authorized users can still upload files successfully, confirming that normal functionality is preserved while preventing unauthorized access.

Part 9) Structured Operation and Security Analysis

Table A — Intended logic, artifacts, and behavior comparison

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 4: Insecure Cloud Configuration	The S3 bucket must only allow uploads from authorized users and must not accept uncontrolled input. Uploaded files must be validated before processing.	S3 bucket configuration, AWS Console (Permissions tab), AWS CLI commands, DVSA lesson description	Uploading a normal file (test.txt) works for authorized users and is processed correctly	Uploading a file with crafted name (test;ls.raw) succeeds, showing attacker-controlled input is accepted and can be processed

Table B — Deviation analysis, fix, and verification

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Lesson 4: Insecure Cloud Configuration	The system allows file uploads without strict access control,	Misconfiguration	Restricted S3 bucket access by enabling “Block all public access” and	After fix, bucket access is restricted and uploads are controlled by	<i>[Optional timing result]</i>

	violating the rule that only authorized users should write to the bucket		limiting write permissions (S3 bucket configuration)	authenticated permissions; public access is blocked	
--	--	--	--	---	--

9.1 Intended Logic and Security Rule(s)

The user uploads files through the DVSA application, which stores them in an S3 bucket. The request flows through API Gateway to a Lambda function that processes uploaded files and generates receipts. Under normal conditions, only authorized users should be able to upload files, and the system should validate file names and content before processing. The expected outcome is that only trusted files are stored and processed.

Security rules:

Only authorized users must be allowed to upload files to the S3 bucket

Uploaded file names must not be used directly in command execution

The system must validate all uploaded content before processing

9.2 Evidence Sources and Behavior Trace

Artifacts used:

- AWS S3 Console (bucket and permissions)
- AWS CLI commands (aws s3 cp)
- Screenshots of upload results
- DVSA lesson description

Normal behavior:

- Uploading a normal file (test.txt) succeeds for authorized users

Exploit behavior:

- Uploading a file with crafted name (test;ls.raw) succeeds, showing attacker-controlled input is accepted

Post-fix behavior:

- After enabling “Block all public access”, uploads are restricted to authorized users and public access is blocked

9.3 Deviation Analysis and Classification

The exploit behavior violates the rule that only authorized users should be able to upload files and that user input must be validated before processing. The system allowed file uploads with attacker-

controlled names, which could be used in backend command execution. This is incorrect because it exposes the system to command injection through unsafe handling of file input.

Classification:

Accidental misconfiguration

9.4 Explainable Fix and Post-Fix Validation

The original configuration incorrectly allowed overly permissive access to the S3 bucket. The fix was applied in the S3 bucket configuration by enabling “Block all public access” and restricting write permissions to authorized users only.

After the fix, the same upload test was repeated and access is now controlled based on authentication and permissions. This confirms that unauthorized or public uploads are no longer allowed. Legitimate users can still upload files, ensuring that normal functionality is preserved.

Part 10) Takeaway / Lessons Learned

This vulnerability was caused by assuming that cloud storage resources were securely configured by default. In a serverless environment, misconfigured cloud services such as S3 can expose the entire backend to attack. The key lesson is to always apply the principle of least privilege and restrict access to only what is necessary. Proper validation and secure configuration of cloud resources are essential to prevent similar issues.